File: Makefile

```
FLAGS = -Wall -pthread

all: t0 t1

clean:
	rm -f t0 t1

t0: t0.c
	gcc -o t0 t0.c $(FLAGS)

t1: t1.c
	gcc -o t1 t1.c $(FLAGS)
```

File: t0.c

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;pthread.h&gt;

void *mythread(void *arg) {
    printf(&quot;%s\n&quot;, (char *) arg);
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 1) {
■fprintf(stderr, &quot;usage: main\n&quot;);
■exit(1);
    }

    pthread_t p1, p2;
    printf(&quot;main: begin\n&quot;);
    pthread_create(&amp;p1, NULL, mythread, &quot;A&quot;);
    pthread_create(&amp;p2, NULL, mythread, &quot;B&quot;);
    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf(&quot;main: end\n&quot;);
    return 0;
}
```

File: t1.c

```
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;pthread.h&gt;

int max;
volatile int counter = 0; // shared global variable

void *mythread(void *arg) {
    char *letter = arg;
    int i; // stack (private per thread)
    printf(&quot;%s: begin [addr of i: %p]\n&quot;, letter, &amp;i);
    for (i = 0; i &lt; max; i++) {
■counter = counter + 1; // shared: only one
    }
    printf(&quot;%s: done\n&quot;, letter);
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
■fprintf(stderr, &quot;usage: ./t1 &lt;loopcount&gt;\n&quot;);
■exit(1);
    }
    max = atoi(argv[1]);

    pthread_t p1, p2;
    printf(&quot;main: begin [counter = %d] [%p]\n&quot;, counter, &amp;counter);
    pthread_create(&amp;p1, NULL, mythread, &quot;A&quot;);
    pthread_create(&amp;p2, NULL, mythread, &quot;B&quot;);
    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf(&quot;main: done\n [counter: %d]\n [should: %d]\n&quot;,
■   counter, max*2);
    return 0;
}
```

File: x86.py

```
#! /usr/bin/env python

import sys
import time
import random
from optparse import OptionParser

#
# HELPER
#
def dospace(howmuch):
    for i in range(howmuch):
        print &#x27;%24s&#x27; % &#x27; &#x27;,

# useful instead of assert
def zassert(cond, str):
    if cond == False:
        print &#x27;ABORT::&#x27;, str
        exit(1)
    return

class cpu:
    #
    # INIT: how much memory?
    #
    def __init__(self, memory, memtrace, regtrace, cctrace, compute, verbose):
        #
        # CONSTANTS
        #

        # conditions
        self.COND_GT        = 0
        self.COND_GTE       = 1
        self.COND_LT        = 2
        self.COND_LTE       = 3
        self.COND_EQ        = 4
        self.COND_NEQ       = 5

        # registers in system
        self.REG_ZERO       = 0
        self.REG_AX         = 1
        self.REG_BX         = 2
        self.REG_CX         = 3
        self.REG_DX         = 4
        self.REG_SP         = 5
        self.REG_BP         = 6

        # system memory: in KB
        self.max_memory     = memory * 1024

        # which memory addrs and registers to trace?
        self.memtrace       = memtrace
        self.regtrace       = regtrace
        self.cctrace        = cctrace
        self.compute        = compute
        self.verbose        = verbose

        self.PC             = 0
        self.registers      = {}
        self.conditions     = {}
        self.labels         = {}
        self.vars           = {}
        self.memory         = {}
        self.pmemory        = {}  # for printable version of what&#x27;s in memory (instructions)

        self.condlist       = [self.COND_GTE, self.COND_GT, self.COND_LTE, self.COND_LT, self.COND_NEQ, self.CON
        self.regnums        = [self.REG_ZERO, self.REG_AX,  self.REG_BX,   self.REG_CX,  self.REG_DX,   self.REG
        self.regnames         = {}
```

```
    self.regnames[&#x27;zero&#x27;] = self.REG_ZERO # hidden zero-valued register
    self.regnames[&#x27;ax&#x27;]   = self.REG_AX
    self.regnames[&#x27;bx&#x27;]   = self.REG_BX
    self.regnames[&#x27;cx&#x27;]   = self.REG_CX
    self.regnames[&#x27;dx&#x27;]   = self.REG_DX
    self.regnames[&#x27;sp&#x27;]   = self.REG_SP
    self.regnames[&#x27;bp&#x27;]   = self.REG_BP

    tmplist = []
    for r in self.regtrace:
        zassert(r in self.regnames, &#x27;Register %s cannot be traced because it does not exist&#x27; % r)
        tmplist.append(self.regnames[r])
    self.regtrace = tmplist

    self.init_memory()
    self.init_registers()
    self.init_condition_codes()

#
# BEFORE MACHINE RUNS
#
def init_condition_codes(self):
    for c in self.condlist:
        self.conditions[c] = False

def init_memory(self):
    for i in range(self.max_memory):
        self.memory[i] = 0

def init_registers(self):
    for i in self.regnums:
        self.registers[i] = 0

def dump_memory(self):
    print &#x27;MEMORY DUMP&#x27;
    for i in range(self.max_memory):
        if i not in self.pmemory and i in self.memory and self.memory[i] != 0:
            print &#x27;  m[%d]&#x27; % i, self.memory[i]

#
# INFORMING ABOUT THE HARDWARE
#
def get_regnum(self, name):
    assert(name in self.regnames)
    return self.regnames[name]

def get_regname(self, num):
    assert(num in self.regnums)
    for rname in self.regnames:
        if self.regnames[rname] == num:
            return rname
    return &#x27;&#x27;

def get_regnums(self):
    return self.regnums

def get_condlist(self):
    return self.condlist

def get_reg(self, reg):
    assert(reg in self.regnums)
    return self.registers[reg]

def get_cond(self, cond):
    assert(cond in self.condlist)
    return self.conditions[cond]

def get_pc(self):
    return self.PC
def set_reg(self, reg, value):
```

```
    assert(reg in self.regnums)
    self.registers[reg] = value

def set_cond(self, cond, value):
    assert(cond in self.condlist)
    self.conditions[cond] = value

def set_pc(self, pc):
    self.PC = pc

#
# INSTRUCTIONS
#
def halt(self):
    return -1

def iyield(self):
    return -2

def nop(self):
    return 0

def rdump(self):
    print &#x27;REGISTERS::&#x27;,
    print &#x27;ax:&#x27;, self.registers[self.REG_AX],
    print &#x27;bx:&#x27;, self.registers[self.REG_BX],
    print &#x27;cx:&#x27;, self.registers[self.REG_CX],
    print &#x27;dx:&#x27;, self.registers[self.REG_DX],

def mdump(self, index):
    print &#x27;m[%d] &#x27; % index, self.memory[index]

def move_i_to_r(self, src, dst):
    self.registers[dst] = src
    return 0

# memory: value, register, register
def move_i_to_m(self, src, value, reg1, reg2):
    tmp = value + self.registers[reg1] + self.registers[reg2]
    self.memory[tmp] = src
    return 0

def move_m_to_r(self, value, reg1, reg2, dst):
    tmp = value + self.registers[reg1] + self.registers[reg2]
    # print &#x27;doing mov&#x27;, &#x27;val:&#x27;, value, &#x27;r1:&#x27;, self.get_regname(reg1), self.re
    self.registers[dst] = self.memory[tmp]

def move_r_to_m(self, src, value, reg1, reg2):
    tmp = value + self.registers[reg1] + self.registers[reg2]
    self.memory[tmp] = self.registers[src]
    return 0

def move_r_to_r(self, src, dst):
    self.registers[dst] = self.registers[src]
    return 0

def add_i_to_r(self, src, dst):
    self.registers[dst] += src
    return 0

def add_r_to_r(self, src, dst):
    self.registers[dst] += self.registers[src]
    return 0

def sub_i_to_r(self, src, dst):
    self.registers[dst] -= src
    return 0

def sub_r_to_r(self, src, dst):
    self.registers[dst] -= self.registers[src]
```

```
    return 0


#
# SUPPORT FOR LOCKS
#
def atomic_exchange(self, src, value, reg1, reg2):
    tmp                 = value + self.registers[reg1] + self.registers[reg2]
    old                 = self.memory[tmp]
    self.memory[tmp]    = self.registers[src]
    self.registers[src] = old
    return 0

def fetchadd(self, src, value, reg1, reg2):
    tmp                 = value + self.registers[reg1] + self.registers[reg2]
    old                 = self.memory[tmp]
    self.memory[tmp]    = self.memory[tmp] + self.registers[src]
    self.registers[src] = old

#
# TEST for conditions
#
def test_all(self, src, dst):
    self.init_condition_codes()
    if dst &gt; src:
        self.conditions[self.COND_GT]  = True
    if dst &gt;= src:
        self.conditions[self.COND_GTE] = True
    if dst &lt; src:
        self.conditions[self.COND_LT]  = True
    if dst &lt;= src:
        self.conditions[self.COND_LTE] = True
    if dst == src:
        self.conditions[self.COND_EQ]  = True
    if dst != src:
        self.conditions[self.COND_NEQ] = True
    return 0

def test_i_r(self, src, dst):
    self.init_condition_codes()
    return self.test_all(src, self.registers[dst])

def test_r_i(self, src, dst):
    self.init_condition_codes()
    return self.test_all(self.registers[src], dst)

def test_r_r(self, src, dst):
    self.init_condition_codes()
    return self.test_all(self.registers[src], self.registers[dst])

#
# JUMPS
#
def jump(self, targ):
    self.PC = targ
    return 0

def jump_notequal(self, targ):
    if self.conditions[self.COND_NEQ] == True:
        self.PC = targ
    return 0

def jump_equal(self, targ):
    if self.conditions[self.COND_EQ] == True:
        self.PC = targ
    return 0

def jump_lessthan(self, targ):
    if self.conditions[self.COND_LT] == True:
        self.PC = targ
```

```
    return 0

def jump_lessthanorequal(self, targ):
    if self.conditions[self.COND_LTE] == True:
        self.PC = targ
    return 0

def jump_greaterthan(self, targ):
    if self.conditions[self.COND_GT] == True:
        self.PC = targ
    return 0

def jump_greaterthanorequal(self, targ):
    if self.conditions[self.COND_GTE] == True:
        self.PC = targ
    return 0

#
# CALL and RETURN
#
def call(self, targ):
    self.registers[self.REG_SP] -= 4
    self.memory[self.registers[self.REG_SP]] = self.PC
    self.PC = targ

def ret(self):
    self.PC = self.memory[self.registers[self.REG_SP]]
    self.registers[self.REG_SP] += 4

#
# STACK and related
#
def push_r(self, reg):
    self.registers[self.REG_SP] -= 4
    self.memory[self.registers[self.REG_SP]] = self.registers[reg]
    return 0

def push_m(self, value, reg1, reg2):
    # print &#x27;push_m&#x27;, value, reg1, reg2
    self.registers[self.REG_SP] -= 4
    tmp = value + self.registers[reg1] + self.registers[reg2]
    # push address onto stack, not memory value itself
    self.memory[self.registers[self.REG_SP]] = tmp
    return 0

def pop(self):
    self.registers[self.REG_SP] += 4

def pop_r(self, dst):
    self.registers[dst] = self.registers[self.REG_SP]
    self.registers[self.REG_SP] += 4

#
# HELPER func for getarg
#
def register_translate(self, r):
    if r in self.regnames:
        return self.regnames[r]
    zassert(False, &#x27;Register %s is not a valid register&#x27; % r)
    return

#
# HELPER in parsing mov (quite primitive) and other ops
# returns: (value, type)
# where type is (TYPE_REGISTER, TYPE_IMMEDIATE, TYPE_MEMORY)
#
# FORMATS
#    %ax           - register
#    $10           - immediate
#    10            - direct memory
```

```
#    10(%ax)       - memory + reg indirect
#    10(%ax,%bx)   - memory + 2 reg indirect
#    10(%ax,%bx,4) - XXX (not handled)
#
def getarg(self, arg):
    tmp1 = arg.replace(&#x27;,&#x27;, &#x27;&#x27;)
    tmp  = tmp1.replace(&#x27; \t&#x27;, &#x27;&#x27;)

    if tmp[0] == &#x27;$&#x27;:
        zassert(len(tmp) == 2, &#x27;correct form is $number (not %s)&#x27; % tmp)
        value = tmp.split(&#x27;$&#x27;)[1]
        zassert(value.isdigit(), &#x27;value [%s] must be a digit&#x27; % value)
        return int(value), &#x27;TYPE_IMMEDIATE&#x27;
    elif tmp[0] == &#x27;%&#x27;:
        register = tmp.split(&#x27;%&#x27;)[1]
        return self.register_translate(register), &#x27;TYPE_REGISTER&#x27;
    elif tmp[0] == &#x27;(&#x27;:
        register = tmp.split(&#x27;(&#x27;)[1].split(&#x27;)&#x27;)[0].split(&#x27;%&#x27;)[1]
        return &#x27;%d,%d,%d&#x27; % (0, self.register_translate(register), self.register_translate(&#x27;z
    elif tmp[0] == &#x27;.&#x27;:
        targ = tmp
        return targ, &#x27;TYPE_LABEL&#x27;
    elif tmp[0].isalpha() and not tmp[0].isdigit():
        zassert(tmp in self.vars, &#x27;Variable %s is not declared&#x27; % tmp)
        # print &#x27;%d,%d,%d&#x27; % (self.vars[tmp], self.register_translate(&#x27;zero&#x27;), self.regi
        return &#x27;%d,%d,%d&#x27; % (self.vars[tmp], self.register_translate(&#x27;zero&#x27;), self.regis
    elif tmp[0].isdigit() or tmp[0] == &#x27;-&#x27;:
        # MOST GENERAL CASE: number(reg,reg) or number(reg)
        # we ignore the common x86 number(reg,reg,constant) for now
        neg = 1
        if tmp[0] == &#x27;-&#x27;:
            tmp = tmp[1:]
            neg = -1
        s = tmp.split(&#x27;(&#x27;)
        if len(s) == 1:
            value = neg * int(tmp)
            # print &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(&#x27;zero&#x27;), self.regi
            return &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(&#x27;zero&#x27;), self.regis
        elif len(s) == 2:
            value = neg * int(s[0])
            t = s[1].split(&#x27;)&#x27;)[0].split(&#x27;,&#x27;)
            if len(t) == 1:
                register = t[0].split(&#x27;%&#x27;)[1]
                # print &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(register), self.register
                return &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(register), self.register_
            elif len(t) == 2:
                register1 = t[0].split(&#x27;%&#x27;)[1]
                register2 = t[1].split(&#x27;%&#x27;)[1]
                # print &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(register1), self.registe
                return &#x27;%d,%d,%d&#x27; % (int(value), self.register_translate(register1), self.register
        else:
            print &#x27;mov: bad argument [%s]&#x27; % tmp
            exit(1)
            return
    zassert(True, &#x27;mov: bad argument [%s]&#x27; % arg)
    return

#
# LOAD a program into memory
# make it ready to execute
#
def load(self, infile, loadaddr):
    pc   = int(loadaddr)
    fd   = open(infile)

    bpc  = loadaddr
    data = 100

    for line in fd:
        cline = line.rstrip()
```

```
    # print &#x27;PASS 1&#x27;, cline

    # remove everything after the comment marker
    ctmp = cline.split(&#x27;#&#x27;)
    assert(len(ctmp) == 1 or len(ctmp) == 2)
    if len(ctmp) == 2:
        cline = ctmp[0]

    # remove empty lines, and split line by spaces
    tmp = cline.split()
    if len(tmp) == 0:
        continue

    # only pay attention to labels and variables
    if tmp[0] == &#x27;.var&#x27;:
        assert(len(tmp) == 2)
        assert(tmp[0] not in self.vars)
        self.vars[tmp[1]] = data
        data += 4
        zassert(data &lt; bpc, &#x27;Load address overrun by static data&#x27;)
        if self.verbose: print &#x27;ASSIGN VAR&#x27;, tmp[0], &quot;--&gt;&quot;, tmp[1], self.vars[tmp
    elif tmp[0][0] == &#x27;.&#x27;:
        assert(len(tmp) == 1)
        self.labels[tmp[0]] = int(pc)
        if self.verbose: print &#x27;ASSIGN LABEL&#x27;, tmp[0], &quot;--&gt;&quot;, pc
    else:
        pc += 1
fd.close()

if self.verbose: print &#x27;&#x27;

# second pass: do everything else
pc = int(loadaddr)
fd = open(infile)
for line in fd:
    cline = line.rstrip()
    # print &#x27;PASS 2&#x27;, cline

    # remove everything after the comment marker
    ctmp = cline.split(&#x27;#&#x27;)
    assert(len(ctmp) == 1 or len(ctmp) == 2)
    if len(ctmp) == 2:
        cline = ctmp[0]

    # remove empty lines, and split line by spaces
    tmp = cline.split()
    if len(tmp) == 0:
        continue

    # skip labels: all else must be instructions
    if cline[0] != &#x27;.&#x27;:
        tmp              = cline.split(None, 1)
        opcode           = tmp[0]
        self.pmemory[pc] = cline.strip()

        # MAIN OPCODE LOOP
        if opcode == &#x27;mov&#x27;:
            rtmp = tmp[1].split(&#x27;,&#x27;, 1)
            zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;mov: needs two args, separated by commas [%s
            arg1 = rtmp[0].strip()
            arg2 = rtmp[1].strip()
            (src, stype) = self.getarg(arg1)
            (dst, dtype) = self.getarg(arg2)
            # print &#x27;MOV&#x27;, src, stype, dst, dtype
            if stype == &#x27;TYPE_MEMORY&#x27; and dtype == &#x27;TYPE_MEMORY&#x27;:
                print &#x27;bad mov: two memory arguments&#x27;
                exit(1)
            elif stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_IMMEDIATE&#x27;:
                print &#x27;bad mov: two immediate arguments&#x27;
                exit(1)
```

```
    elif stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc]  = &#x27;self.move_i_to_r(%d, %d)&#x27; % (int(src), dst)
    elif stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc]  = &#x27;self.move_i_to_r(%d, %d)&#x27; % (int(src), dst)
    elif stype == &#x27;TYPE_MEMORY&#x27;    and dtype == &#x27;TYPE_REGISTER&#x27;:
        tmp = src.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        self.memory[pc] = &#x27;self.move_m_to_r(%d, %d, %d, %d)&#x27; % (int(tmp[0]), int(tmp[1
    elif stype == &#x27;TYPE_REGISTER&#x27;  and dtype == &#x27;TYPE_MEMORY&#x27;:
        tmp = dst.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        self.memory[pc] = &#x27;self.move_r_to_m(%d, %d, %d, %d)&#x27; % (src, int(tmp[0]), int(
    elif stype == &#x27;TYPE_REGISTER&#x27;  and dtype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc] = &#x27;self.move_r_to_r(%d, %d)&#x27; % (src, dst)
    elif stype == &#x27;TYPE_IMMEDIATE&#x27;  and dtype == &#x27;TYPE_MEMORY&#x27;:
        tmp = dst.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        self.memory[pc] = &#x27;self.move_i_to_m(%d, %d, %d, %d)&#x27; % (src, int(tmp[0]), int(
    else:
        zassert(False, &#x27;malformed mov instruction&#x27;)
elif opcode == &#x27;pop&#x27;:
    if len(tmp) == 1:
        self.memory[pc] = &#x27;self.pop()&#x27;
    elif len(tmp) == 2:
        arg = tmp[1].strip()
        (dst, dtype) = self.getarg(arg)
        zassert(dtype == &#x27;TYPE_REGISTER&#x27;, &#x27;Can only pop into a register&#x27;)
        self.memory[pc] = &#x27;self.pop_r(%d)&#x27; % dst
    else:
        zassert(False, &#x27;pop instruction must take zero/one args&#x27;)
elif opcode == &#x27;push&#x27;:
    (src, stype) = self.getarg(tmp[1].strip())
    if stype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc] = &#x27;self.push_r(%d)&#x27; % (int(src))
    elif stype == &#x27;TYPE_MEMORY&#x27;:
        tmp = src.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        self.memory[pc] = &#x27;self.push_m(%d,%d,%d)&#x27; % (int(tmp[0]), int(tmp[1]), int(tmp
    else:
        zassert(False, &#x27;Cannot push anything but registers&#x27;)
elif opcode == &#x27;call&#x27;:
    (targ, ttype) = self.getarg(tmp[1].strip())
    if ttype == &#x27;TYPE_LABEL&#x27;:
        self.memory[pc] = &#x27;self.call(%d)&#x27; % (int(self.labels[targ]))
    else:
        zassert(False, &#x27;Cannot call anything but a label&#x27;)
elif opcode == &#x27;ret&#x27;:
    assert(len(tmp) == 1)
    self.memory[pc] = &#x27;self.ret()&#x27;
elif opcode == &#x27;add&#x27;:
    rtmp = tmp[1].split(&#x27;,&#x27;, 1)
    zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;add: needs two args, separated by commas [%s
    arg1 = rtmp[0].strip()
    arg2 = rtmp[1].strip()
    (src, stype) = self.getarg(arg1)
    (dst, dtype) = self.getarg(arg2)
    if stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc] = &#x27;self.add_i_to_r(%d, %d)&#x27; % (int(src), dst)
    elif stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
        self.memory[pc] = &#x27;self.add_r_to_r(%d, %d)&#x27; % (int(src), dst)
    else:
        zassert(False, &#x27;malformed usage of add instruction&#x27;)
elif opcode == &#x27;sub&#x27;:
    rtmp = tmp[1].split(&#x27;,&#x27;, 1)
    zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;sub: needs two args, separated by commas [%s
    arg1 = rtmp[0].strip()
    arg2 = rtmp[1].strip()
    (src, stype) = self.getarg(arg1)
    (dst, dtype) = self.getarg(arg2)
    if stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
```

```
            self.memory[pc] = &#x27;self.sub_i_to_r(%d, %d)&#x27; % (int(src), dst)
        elif stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
            self.memory[pc] = &#x27;self.sub_r_to_r(%d, %d)&#x27; % (int(src), dst)
        else:
            zassert(False, &#x27;malformed usage of sub instruction&#x27;)
    elif opcode == &#x27;fetchadd&#x27;:
        rtmp = tmp[1].split(&#x27;,&#x27;, 1)
        zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;fetchadd: needs two args, separated by comma
        arg1 = rtmp[0].strip()
        arg2 = rtmp[1].strip()
        (src, stype) = self.getarg(arg1)
        (dst, dtype) = self.getarg(arg2)
        tmp = dst.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        if stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_MEMORY&#x27;:
            self.memory[pc] = &#x27;self.fetchadd(%d, %d, %d, %d)&#x27; % (src, int(tmp[0]), int(tmp
        else:
            zassert(False, &#x27;poorly specified fetch and add&#x27;)
    elif opcode == &#x27;xchg&#x27;:
        rtmp = tmp[1].split(&#x27;,&#x27;, 1)
        zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;xchg: needs two args, separated by commas [%
        arg1 = rtmp[0].strip()
        arg2 = rtmp[1].strip()
        (src, stype) = self.getarg(arg1)
        (dst, dtype) = self.getarg(arg2)
        tmp = dst.split(&#x27;,&#x27;)
        assert(len(tmp) == 3)
        if stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_MEMORY&#x27;:
            self.memory[pc] = &#x27;self.atomic_exchange(%d, %d, %d, %d)&#x27; % (src, int(tmp[0]),
        else:
            zassert(False, &#x27;poorly specified atomic exchange&#x27;)
    elif opcode == &#x27;test&#x27;:
        rtmp = tmp[1].split(&#x27;,&#x27;, 1)
        zassert(len(tmp) == 2 and len(rtmp) == 2, &#x27;test: needs two args, separated by commas [%
        arg1 = rtmp[0].strip()
        arg2 = rtmp[1].strip()
        (src, stype) = self.getarg(arg1)
        (dst, dtype) = self.getarg(arg2)
        if stype == &#x27;TYPE_IMMEDIATE&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
            self.memory[pc] = &#x27;self.test_i_r(%d, %d)&#x27; % (int(src), dst)
        elif stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_REGISTER&#x27;:
            self.memory[pc] = &#x27;self.test_r_r(%d, %d)&#x27; % (int(src), dst)
        elif stype == &#x27;TYPE_REGISTER&#x27; and dtype == &#x27;TYPE_IMMEDIATE&#x27;:
            self.memory[pc] = &#x27;self.test_r_i(%d, %d)&#x27; % (int(src), dst)
        else:
            zassert(False, &#x27;malformed usage of test instruction&#x27;)
    elif opcode == &#x27;j&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump(%d)&#x27; % int(self.labels[targ])
    elif opcode == &#x27;jne&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump_notequal(%d)&#x27; % int(self.labels[targ])
    elif opcode == &#x27;je&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump_equal(%d)&#x27; % self.labels[targ]
    elif opcode == &#x27;jlt&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump_lessthan(%d)&#x27; % int(self.labels[targ])
    elif opcode == &#x27;jlte&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump_lessthanorequal(%s)&#x27; % self.labels[targ]
    elif opcode == &#x27;jgt&#x27;:
        (targ, ttype) = self.getarg(tmp[1].strip())
        zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
        self.memory[pc] = &#x27;self.jump_greaterthan(%d)&#x27; % int(self.labels[targ])
```

```
            elif opcode == &#x27;jgte&#x27;:
                (targ, ttype) = self.getarg(tmp[1].strip())
                zassert(ttype == &#x27;TYPE_LABEL&#x27;, &#x27;bad jump target [%s]&#x27; % tmp[1].strip())
                self.memory[pc] = &#x27;self.jump_greaterthanorequal(%s)&#x27; % self.labels[targ]
            elif opcode == &#x27;nop&#x27;:
                self.memory[pc] = &#x27;self.nop()&#x27;
            elif opcode == &#x27;halt&#x27;:
                self.memory[pc] = &#x27;self.halt()&#x27;
            elif opcode == &#x27;yield&#x27;:
                self.memory[pc] = &#x27;self.iyield()&#x27;
            elif opcode == &#x27;rdump&#x27;:
                self.memory[pc] = &#x27;self.rdump()&#x27;
            elif opcode == &#x27;mdump&#x27;:
                self.memory[pc] = &#x27;self.mdump(%s)&#x27; % tmp[1]
            else:
                print &#x27;illegal opcode: &#x27;, opcode
                exit(1)

            if self.verbose: print &#x27;pc:%d LOADING %20s --&gt; %s&#x27; % (pc, self.pmemory[pc], self.me

            # INCREMENT PC for loader
            pc += 1
        # END: loop over file
        fd.close()
        if self.verbose: print &#x27;&#x27;
        return
    # END: load

    def print_headers(self, procs):
        # print some headers
        if len(self.memtrace) &gt; 0:
            for m in self.memtrace:
                if m[0].isdigit():
                    print &#x27;%5d&#x27; % int(m),
                else:
                    zassert(m in self.vars, &#x27;Traced variable %s not declared&#x27; % m)
                    print &#x27;%5s&#x27; % m,
            print &#x27; &#x27;,
        if len(self.regtrace) &gt; 0:
            for r in self.regtrace:
                print &#x27;%5s&#x27; % self.get_regname(r),
            print &#x27; &#x27;,
        if cctrace == True:
            print &#x27;&gt;= &gt;  &lt;= &lt;  != ==&#x27;,

        # and per thread
        for i in range(procs.getnum()):
            print &#x27;       Thread %d        &#x27; % i,
        print &#x27;&#x27;
        return

    def print_trace(self, newline):
        if len(self.memtrace) &gt; 0:
            for m in self.memtrace:
                if self.compute:
                    if m[0].isdigit():
                        print &#x27;%5d&#x27; % self.memory[int(m)],
                    else:
                        zassert(m in self.vars, &#x27;Traced variable %s not declared&#x27; % m)
                        print &#x27;%5d&#x27; % self.memory[self.vars[m]],
                else:
                    print &#x27;%5s&#x27; % &#x27;?&#x27;,
            print &#x27; &#x27;,
        if len(self.regtrace) &gt; 0:
            for r in self.regtrace:
                if self.compute:
                    print &#x27;%5d&#x27; % self.registers[r],
                else:
                    print &#x27;%5s&#x27; % &#x27;?&#x27;,
            print &#x27; &#x27;,
```

```
    if cctrace == True:
        for c in self.condlist:
            if self.compute:
                if self.conditions[c]:
                    print &#x27;1 &#x27;,
                else:
                    print &#x27;0 &#x27;,
            else:
                print &#x27;? &#x27;,
    if (len(self.memtrace) &gt; 0 or len(self.regtrace) &gt; 0 or cctrace == True) and newline == True:
        print &#x27;&#x27;
    return

def setint(self, intfreq, intrand):
    if intrand == False:
        return intfreq
    return int(random.random() * intfreq) + 1

def run(self, procs, intfreq, intrand):
    # hw init: cc&#x27;s, interrupt frequency, etc.
    interrupt = self.setint(intfreq, intrand)
    icount    = 0

    self.print_headers(procs)
    self.print_trace(True)

    while True:
        # need thread ID of current process
        tid = procs.getcurr().gettid()

        # FETCH
        prevPC       = self.PC
        instruction  = self.memory[self.PC]
        self.PC     += 1

        # DECODE and EXECUTE
        # key: self.PC may be changed during eval; thus MUST be incremented BEFORE eval
        rc = eval(instruction)

        # tracing details: ALWAYS AFTER EXECUTION OF INSTRUCTION
        self.print_trace(False)

        # output: thread-proportional spacing followed by PC and instruction
        dospace(tid)
        print prevPC, self.pmemory[prevPC]
        icount += 1

        # halt instruction issued
        if rc == -1:
            procs.done()
            if procs.numdone() == procs.getnum():
                return icount
            procs.next()
            procs.restore()

            self.print_trace(False)
            for i in range(procs.getnum()):
                print &#x27;----- Halt;Switch ----- &#x27;,
            print &#x27;&#x27;

        # do interrupt processing
        interrupt -= 1
        if interrupt == 0 or rc == -2:
            interrupt = self.setint(intfreq, intrand)
            procs.save()
            procs.next()
            procs.restore()

            self.print_trace(False)
            for i in range(procs.getnum()):
```

```
                    print &#x27;------ Interrupt ------ &#x27;,
                print &#x27;&#x27;
        # END: while
        return

#
# END: class cpu
#


#
# PROCESS LIST class
#
class proclist:
    def __init__(self):
        self.plist  = []
        self.curr   = 0
        self.active = 0

    def done(self):
        self.plist[self.curr].setdone()
        self.active -= 1

    def numdone(self):
        return len(self.plist) - self.active

    def getnum(self):
        return len(self.plist)

    def add(self, p):
        self.active += 1
        self.plist.append(p)

    def getcurr(self):
        return self.plist[self.curr]

    def save(self):
        self.plist[self.curr].save()

    def restore(self):
        self.plist[self.curr].restore()

    def next(self):
        for i in range(self.curr+1, len(self.plist)):
            if self.plist[i].isdone() == False:
                self.curr = i
                return
        for i in range(0, self.curr+1):
            if self.plist[i].isdone() == False:
                self.curr = i
                return

#
# PROCESS class
#
class process:
    def __init__(self, cpu, tid, pc, stackbottom, reginit):
        self.cpu   = cpu  # object reference
        self.tid   = tid
        self.pc    = pc
        self.regs  = {}
        self.cc    = {}
        self.done  = False
        self.stack = stackbottom

        # init regs: all 0 or specially set to something
        for r in self.cpu.get_regnums():
            self.regs[r] = 0
        if reginit != &#x27;&#x27;:
            # form: ax=1,bx=2 (for some subset of registers)
```

```
            for r in reginit.split(&#x27;:&#x27;):
                tmp = r.split(&#x27;=&#x27;)
                assert(len(tmp) == 2)
                self.regs[self.cpu.get_regnum(tmp[0])] = int(tmp[1])

        # init CCs
        for c in self.cpu.get_condlist():
            self.cc[c] = False

        # stack
        self.regs[self.cpu.get_regnum(&#x27;sp&#x27;)] = stackbottom
        # print &#x27;REG&#x27;, self.cpu.get_regnum(&#x27;sp&#x27;), self.regs[self.cpu.get_regnum(&#x27;sp&#x2

        return

    def gettid(self):
        return self.tid

    def save(self):
        self.pc = self.cpu.get_pc()
        for c in self.cpu.get_condlist():
            self.cc[c] = self.cpu.get_cond(c)
        for r in self.cpu.get_regnums():
            self.regs[r] = self.cpu.get_reg(r)

    def restore(self):
        self.cpu.set_pc(self.pc)
        for c in self.cpu.get_condlist():
            self.cpu.set_cond(c, self.cc[c])
        for r in self.cpu.get_regnums():
            self.cpu.set_reg(r, self.regs[r])

    def setdone(self):
        self.done = True

    def isdone(self):
        return self.done == True

#
# main program
#
parser = OptionParser()
parser.add_option(&#x27;-s&#x27;, &#x27;--seed&#x27;,      default=0,          help=&#x27;the random seed&#x27;,
parser.add_option(&#x27;-t&#x27;, &#x27;--threads&#x27;,   default=2,          help=&#x27;number of threads&#x27
parser.add_option(&#x27;-p&#x27;, &#x27;--program&#x27;,   default=&#x27;&#x27;,         help=&#x27;source progr
parser.add_option(&#x27;-i&#x27;, &#x27;--interrupt&#x27;, default=50,         help=&#x27;interrupt frequency&#x
parser.add_option(&#x27;-r&#x27;, &#x27;--randints&#x27;,  default=False,      help=&#x27;if interrupts are rand
parser.add_option(&#x27;-a&#x27;, &#x27;--argv&#x27;,      default=&#x27;&#x27;,
                  help=&#x27;comma-separated per-thread args (e.g., ax=1,ax=2 sets thread 0 ax reg to 1 and thre
                  action=&#x27;store&#x27;,      type=&#x27;string&#x27;, dest=&#x27;argv&#x27;)
parser.add_option(&#x27;-L&#x27;, &#x27;--loadaddr&#x27;,  default=1000,       help=&#x27;address where to load
parser.add_option(&#x27;-m&#x27;, &#x27;--memsize&#x27;,   default=128,        help=&#x27;size of address space
parser.add_option(&#x27;-M&#x27;, &#x27;--memtrace&#x27;,  default=&#x27;&#x27;,         help=&#x27;comma-separa
                  type=&#x27;string&#x27;, dest=&#x27;memtrace&#x27;)
parser.add_option(&#x27;-R&#x27;, &#x27;--regtrace&#x27;,  default=&#x27;&#x27;,         help=&#x27;comma-separa
                  type=&#x27;string&#x27;, dest=&#x27;regtrace&#x27;)
parser.add_option(&#x27;-C&#x27;, &#x27;--cctrace&#x27;,   default=False,      help=&#x27;should we trace condit
parser.add_option(&#x27;-S&#x27;, &#x27;--printstats&#x27;,default=False,      help=&#x27;print some extra stats
parser.add_option(&#x27;-v&#x27;, &#x27;--verbose&#x27;,   default=False,      help=&#x27;print some extra info&
parser.add_option(&#x27;-c&#x27;, &#x27;--compute&#x27;,   default=False,      help=&#x27;compute answers for me
(options, args) = parser.parse_args()

print &#x27;ARG seed&#x27;,               options.seed
print &#x27;ARG numthreads&#x27;,         options.numthreads
print &#x27;ARG program&#x27;,            options.progfile
print &#x27;ARG interrupt frequency&#x27;, options.intfreq
print &#x27;ARG interrupt randomness&#x27;,options.intrand
print &#x27;ARG argv&#x27;,               options.argv
print &#x27;ARG load address&#x27;,       options.loadaddr
print &#x27;ARG memsize&#x27;,            options.memsize
```

```
print &#x27;ARG memtrace&#x27;,            options.memtrace
print &#x27;ARG regtrace&#x27;,            options.regtrace
print &#x27;ARG cctrace&#x27;,             options.cctrace
print &#x27;ARG printstats&#x27;,          options.printstats
print &#x27;ARG verbose&#x27;,             options.verbose
print &#x27;&#x27;

seed       = int(options.seed)
numthreads = int(options.numthreads)
intfreq    = int(options.intfreq)
zassert(intfreq &gt; 0, &#x27;Interrupt frequency must be greater than 0&#x27;)
intrand    = int(options.intrand)
progfile   = options.progfile
zassert(progfile != &#x27;&#x27;, &#x27;Program file must be specified&#x27;)
argv       = options.argv.split(&#x27;,&#x27;)
zassert(len(argv) == numthreads or len(argv) == 1, &#x27;argv: must be one per-thread or just one set of values

loadaddr   = options.loadaddr
memsize    = options.memsize
random.seed(seed)

memtrace   = []
if options.memtrace != &#x27;&#x27;:
    for m in options.memtrace.split(&#x27;,&#x27;):
        memtrace.append(m)

regtrace   = []
if options.regtrace != &#x27;&#x27;:
    for r in options.regtrace.split(&#x27;,&#x27;):
        regtrace.append(r)

cctrace    = options.cctrace

printstats = options.printstats
verbose    = options.verbose

#
# MAIN program
#
debug = False
debug = False

cpu = cpu(memsize, memtrace, regtrace, cctrace, options.solve, verbose)

# load a program
cpu.load(progfile, loadaddr)

# process list
procs = proclist()
pid   = 0
stack = memsize * 1000
for t in range(numthreads):
    if len(argv) &gt; 1:
        arg = argv[pid]
    else:
        arg = argv[0]
    procs.add(process(cpu, pid, loadaddr, stack, arg))
    stack -= 1000
    pid += 1

# get first one ready!
procs.restore()

# run it
t1 = time.clock()
ic = cpu.run(procs, intfreq, intrand)
t2 = time.clock()

if printstats:
    print &#x27;&#x27;
```

```
    print &#x27;STATS:: Instructions    %d&#x27; % ic
    print &#x27;STATS:: Emulation Rate  %.2f kinst/sec&#x27; % (float(ic) / float(t2 - t1) / 1000.0)

# use this for profiling
# import cProfile
# cProfile.run(&#x27;run()&#x27;)
```

File: simple-race.s

```
.main
# this is a critical section
mov 2000, %ax   # get the value at the address
add $1, %ax ■# increment it
mov %ax, 2000   # store it back
halt
```

File: loop.s

```
.main
.top
sub  $1,%dx
test $0,%dx
jgte .top
halt
```

File: looping-race-nolock.s

```
# assumes %bx has loop count in it

.main
.top
# critical section
mov 2000, %ax  # get &#x27;value&#x27; at address 2000
add $1, %ax    # increment it
mov %ax, 2000  # store it back

# see if we&#x27;re still looping
sub  $1, %bx
test $0, %bx
jgt .top

halt
```